

Oráculos y primeros algoritmos cuánticos

¿Cómo calculamos una función booleana?

Considere una función Booleana $f : \{0, 1\}^n \longrightarrow \{0, 1\}$

Queremos utilizar f dentro de un circuito cuántico

Podríamos intentar representar su evaluación mediante la transformación

$$|x\rangle \longmapsto |f(x)\rangle$$

¿Cómo calculamos una función booleana?

El problema es que f no es una función inyectiva si $n \geq 2$

- Podemos entonces perder información y la computación no sería reversible

Nos gustaría representar f a través de una matriz unitaria U_f

- ¿Cómo podemos calcular $f(x)$ conservando la información de entrada?

El oráculo XOR

El **oráculo XOR** U_f se define de la siguiente forma, suponiendo que $x \in \{0, 1\}^n$ e $y \in \{0, 1\}$:

$$U_f|yx\rangle = |(y \oplus f(x))x\rangle$$

En particular, si inicializamos el qubit y en $|0\rangle$, tenemos que:

$$U_f|0x\rangle = |f(x)x\rangle$$

¿Existe este oráculo para cualquier función booleana?

Aplicando dos veces la transformación, tenemos que:

$$\begin{aligned} & U_f^2 \left((\beta|0\rangle + \gamma|1\rangle) \otimes \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle \right) \\ &= U_f U_f \left(\beta(|0\rangle \otimes \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle) + \gamma(|1\rangle \otimes \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle) \right) \\ &= U_f \left(\beta U_f \sum_{x \in \{0,1\}^n} \alpha_x |0x\rangle + \gamma U_f \sum_{x \in \{0,1\}^n} \alpha_x |1x\rangle \right) \\ &= U_f \left(\beta \sum_{x \in \{0,1\}^n} \alpha_x U_f |0x\rangle + \gamma \sum_{x \in \{0,1\}^n} \alpha_x U_f |1x\rangle \right) \end{aligned}$$

¿Existe este oráculo para cualquier función booleana?

$$\begin{aligned}
 &= U_f \left(\beta \sum_{x \in \{0,1\}^n} \alpha_x |f(x)x\rangle + \gamma \sum_{x \in \{0,1\}^n} \alpha_x |(1 \oplus f(x))x\rangle \right) \\
 &= \beta \sum_{x \in \{0,1\}^n} \alpha_x U_f |f(x)x\rangle + \gamma \sum_{x \in \{0,1\}^n} \alpha_x U_f |(1 \oplus f(x))x\rangle \\
 &= \beta \sum_{x \in \{0,1\}^n} \alpha_x |(f(x) \oplus f(x))x\rangle + \gamma \sum_{x \in \{0,1\}^n} \alpha_x |(1 \oplus f(x) \oplus f(x))x\rangle \\
 &= \beta \sum_{x \in \{0,1\}^n} \alpha_x |0x\rangle + \gamma \sum_{x \in \{0,1\}^n} \alpha_x |1x\rangle \\
 &= \beta |0\rangle \otimes \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle + \gamma |1\rangle \otimes \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle = (\beta |0\rangle + \gamma |1\rangle) \otimes \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle
 \end{aligned}$$

¿Existe este oráculo para cualquier función booleana?

Tenemos que: $U_f^2 = I$

Vamos a demostrar que $U_f^\dagger = U_f$

- De esto concluimos que U_f es una matriz unitaria puesto que $U_f^\dagger U_f = U_f U_f = I$

¿Existe este oráculo para cualquier función booleana?

Considere la siguiente matriz, donde $X^0 = I$ y $X^1 = X$:

$$\sum_{u \in \{0,1\}^n} X^{f(u)} \otimes |u\rangle\langle u|$$

Observe que $X^{f(x)} |y\rangle = |y \oplus f(x)\rangle$ para $x \in \{0,1\}^n$ e $y \in \{0,1\}$

¿Existe este oráculo para cualquier función booleana?

Entonces tenemos lo siguiente para $x \in \{0, 1\}^n$ e $y \in \{0, 1\}$:

$$\begin{aligned} \left(\sum_{u \in \{0,1\}^n} X^{f(u)} \otimes |u\rangle\langle u| \right) (|y\rangle \otimes |x\rangle) &= \sum_{u \in \{0,1\}^n} (X^{f(u)} \otimes |u\rangle\langle u|)(|y\rangle \otimes |x\rangle) \\ &= \sum_{u \in \{0,1\}^n} X^{f(u)} |y\rangle \otimes |u\rangle\langle u|x\rangle \\ &= \sum_{u \in \{0,1\}^n} X^{f(u)} |y\rangle \otimes |u\rangle\langle u | x\rangle \\ &= X^{f(x)} |y\rangle \otimes |x\rangle = |y \oplus f(x)\rangle \otimes |x\rangle = U_f(|y\rangle \otimes |x\rangle) \end{aligned}$$

¿Existe este oráculo para cualquier función booleana?

Concluimos que:
$$U_f = \sum_{u \in \{0,1\}^n} X^{f(u)} \otimes |u\rangle\langle u|$$

Por lo tanto:

$$\begin{aligned} U_f^\dagger &= \left(\sum_{u \in \{0,1\}^n} X^{f(u)} \otimes |u\rangle\langle u| \right)^\dagger \\ &= \sum_{u \in \{0,1\}^n} (X^{f(u)} \otimes |u\rangle\langle u|)^\dagger \\ &= \sum_{u \in \{0,1\}^n} (X^{f(u)})^\dagger \otimes (|u\rangle\langle u|)^\dagger = \sum_{u \in \{0,1\}^n} X^{f(u)} \otimes |u\rangle\langle u| = U_f \end{aligned}$$

¿Existe este oráculo para cualquier función booleana?

Tenemos entonces que la matriz U_f es unitaria

Concluimos que toda función booleana $f : \{0, 1\}^n \rightarrow \{0, 1\}$ admite un oráculo XOR unitario

- f admite un oráculo que puede ser implementado
- ¿Cómo construimos este operador usando las compuertas que conocemos?

Consultar una función

En una consulta clásica elegimos una entrada $x \in \{0, 1\}^n$ y recibimos $f(x)$

En una consulta cuántica elegimos $\beta|0\rangle + \gamma|1\rangle$ y

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle$$

y recibimos $U_f((\beta|0\rangle + \gamma|1\rangle) \otimes |\psi\rangle)$

Para analizar el costo de un algoritmo necesitamos estudiar cuántas veces se accede a la función

Consultar una función

En particular, tenemos que:

$$U_f (|0\rangle \otimes |\psi\rangle) = \sum_{x \in \{0,1\}^n} \alpha_x |f(x)x\rangle$$

Al medir ambos registros, obtenemos el par $(f(x), x)$ con probabilidad $|\alpha_x|^2$, para $x \in \{0, 1\}^n$

- Queremos entonces aumentar la amplitud de los resultados que necesitamos

Un oráculo para AND

Considere $f(x_1, x_0) = x_1 \wedge x_0$

Queremos construir el oráculo $|yx_1x_0\rangle \longmapsto |(y \oplus (x_1 \wedge x_0))x_1x_0\rangle$

¿Cómo hacemos esto?

- Usamos una compuerta de Toffoli

Un oráculo para NOT

Considere $f(x_0) = \neg x_0$

Queremos construir el oráculo $|yx_0\rangle \mapsto |(y \oplus \neg x_0)x_0\rangle$

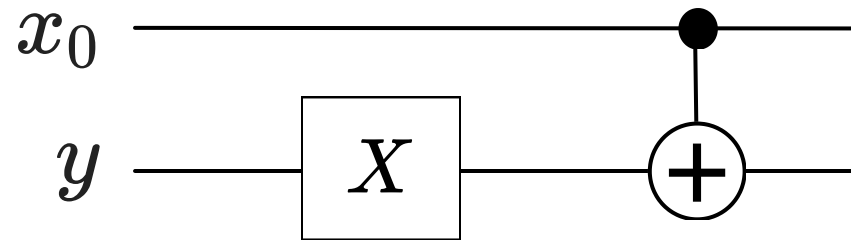
¿Cómo hacemos esto?

- Podemos usar la equivalencia: $y \oplus \neg x_0 \equiv y \oplus (1 \oplus x_0) \equiv (y \oplus 1) \oplus x_0$
- ¿Qué compuertas necesitamos para implementar esto?

Un oráculo para NOT

Considere $f(x_0) = \neg x_0$

Queremos construir el oráculo $|yx_0\rangle \mapsto |(y \oplus \neg x_0)x_0\rangle$



Un oráculo para OR

Considere $f(x_1, x_0) = x_1 \vee x_0$

Queremos construir el oráculo $|yx_1x_0\rangle \longmapsto |(y \oplus (x_1 \vee x_0))x_1x_0\rangle$

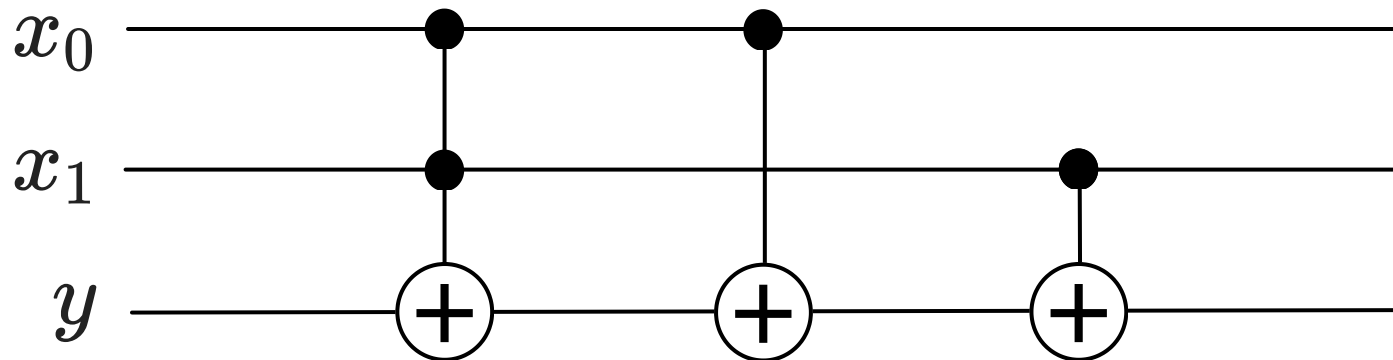
¿Cómo hacemos esto?

- Podemos usar la equivalencia: $x_1 \vee x_0 \equiv x_1 \oplus x_0 \oplus (x_1 \wedge x_0)$
- ¿Qué compuertas necesitamos para implementar esto?

Un oráculo para OR

Considere $f(x_1, x_0) = x_1 \vee x_0$

Queremos construir el oráculo $|yx_1x_0\rangle \mapsto |(y \oplus x_1 \oplus x_0 \oplus (x_1 \wedge x_0))x_1x_0\rangle$



Un oráculo para AND generalizado

Considere $f(x_3, x_2, x_1, x_0) = x_3 \wedge x_2 \wedge x_1 \wedge x_0$

Ejercicio: construya un oráculo XOR para esta función

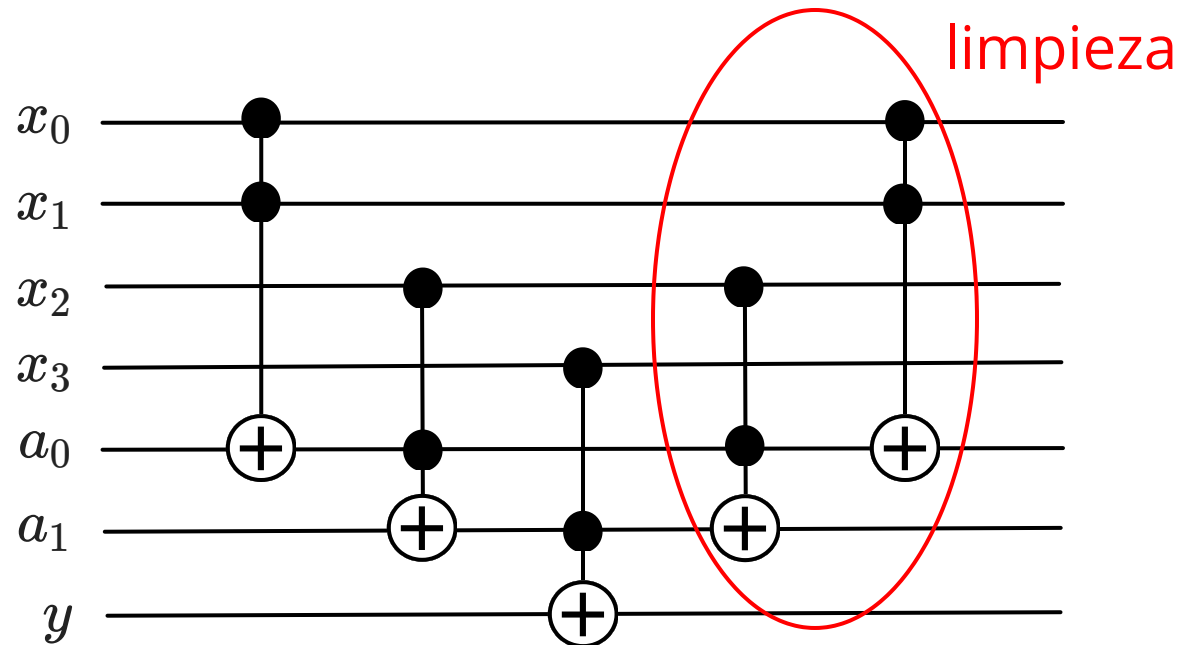
Solución:

Un oráculo para AND generalizado

Considere $f(x_3, x_2, x_1, x_0) = x_3 \wedge x_2 \wedge x_1 \wedge x_0$

Ejercicio: construya un oráculo XOR para esta función

Solución:



Un oráculo para AND generalizado

En este caso el oráculo tiene dos qubits auxiliares: a_1 y a_0

- El oráculo realiza entonces la siguiente transformación considerando el valor 0 en los qubits auxiliares:

$$|y00x_3x_2x_1x_0\rangle \longmapsto |(y \oplus (x_3 \wedge x_2 \wedge x_1 \wedge x_0))00x_3x_2x_1x_0\rangle$$

Ejercicio: construya un oráculo XOR para la función

$$f(x_{n-1}, \dots, x_0) = \bigwedge_{i=0}^{n-1} x_i$$

Un oráculo que reconoce una entrada

Para una cadena fija $a \in \{0, 1\}^n$, definimos

$$m_a(x) = \begin{cases} 1 & x = a \\ 0 & x \neq a \end{cases}$$

Su oráculo debe realizar $U_{m_a}|yx\rangle = |(y \oplus m_a(x))x\rangle$

Ejercicio: Construya un oráculo para $a = (1, 1, 0)$

Un oráculo que reconoce una entrada

Para una cadena fija $a \in \{0, 1\}^n$, definimos

$$m_a(x) = \begin{cases} 1 & x = a \\ 0 & x \neq a \end{cases}$$

Su oráculo debe realizar $U_{m_a}|yx\rangle = |(y \oplus m_a(x))x\rangle$

En este caso tenemos que $m_a(x_2, x_1, x_0) = x_2 \wedge x_1 \wedge \neg x_0$

Solución:

Vamos a construir un oráculo que realiza lo siguiente:

$$|y0x_2x_1x_0\rangle \longmapsto |(y \oplus m_a(x_2, x_1, x_0))0x_2x_1x_0\rangle$$

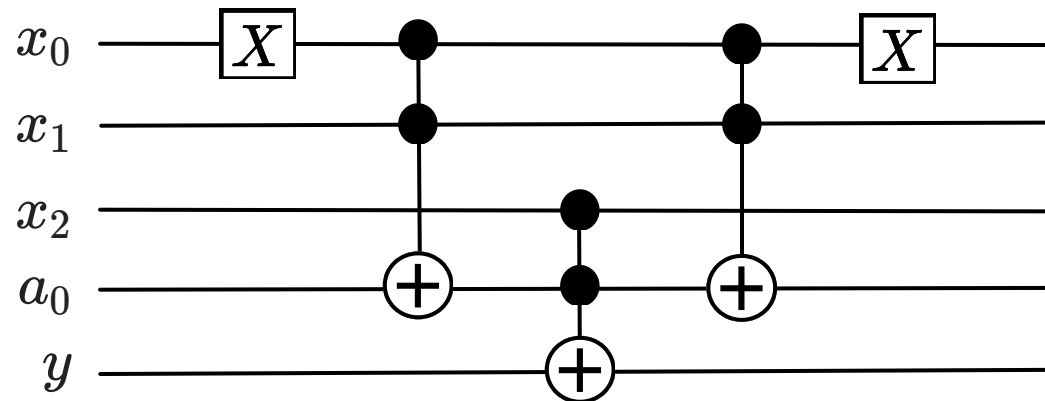
Un oráculo que reconoce una entrada

Para una cadena fija $a \in \{0, 1\}^n$, definimos

$$m_a(x) = \begin{cases} 1 & x = a \\ 0 & x \neq a \end{cases}$$

Su oráculo debe realizar $U_{m_a}|yx\rangle = |(y \oplus m_a(x))x\rangle$

Solución:



Construcción para una función booleana arbitraria

Para una función $f : \{0, 1\}^n \rightarrow \{0, 1\}$ tenemos que:

$$f(x) = \bigoplus_{a \in \{0,1\}^n : f(a)=1} m_a(x)$$

Ejercicio: Explique cómo esta representación puede ser utilizada para construir U_f , considerando que ya sabemos construir U_{m_a}

Solución del ejercicio en Qiskit

Vamos a mostrar cómo la siguiente representación puede ser utilizada para construir U_f para una función Booleana $f : \{0, 1\}^n \rightarrow \{0, 1\}$:

$$f(x) = \bigoplus_{a \in \{0,1\}^n : f(a)=1} m_a(x)$$

Para la construcción en Qiskit de U_f suponemos que $n \geq 3$

Solución del ejercicio en Qiskit

```
from qiskit import QuantumCircuit
import numpy as np

def circuito_entrada(entrada):
    n = len(entrada)
    circuito = QuantumCircuit(2*n-1)
    for i in range(n):
        if entrada[i] == 0:
            circuito.x(n-1-i)
    circuito.ccx(0,1,n)
    for i in range(2,n):
        circuito.ccx(i,i+n-2,i+n-1)
    for i in range(n-2,1,-1):
        circuito.ccx(i,i+n-2,i+n-1)
    circuito.ccx(0,1,n)
    for i in range(n):
        if entrada[i] == 0:
            circuito.x(n-i-1)
    return(circuito)
```

Recuerde que el orden de los bits de entrada desde más a menos significativo es entrada[0], entrada[1], ..., entrada[n-1]

Solución del ejercicio en Qiskit

```
def circuito_tabla(tabla, dim):  
    circuito = QuantumCircuit(2*dim-1)  
    for i in range(len(tabla)):  
        circuito = circuito.compose(circuito_entrada(tabla[i]))  
        if i < len(tabla)-1:  
            circuito.barrier()  
    return circuito
```

Solución del ejercicio en Qiskit

```
def circuito_tabla(tabla, dim):  
    circuito = QuantumCircuit(2*dim-  
    for i in range(len(tabla)):  
        circuito = circuito.compose(  
            if i < len(tabla)-1:  
                circuito.barrier()  
    return circuito
```

Contiene las entradas
donde la función
Booleana vale 1,
sin repeticiones

Solución del ejercicio en Qiskit

```
def circuito_tabla(tabla, dim):  
    circuito = QuantumCircuit(2*dim-1)  
    for i in range(len(tabla)):  
        circuito = circuito.compose(circuito_entrada(tabla[i]))  
        if i < len(tabla)-1:  
            circuito.barrier()  
    return circuito
```

Separa las implementaciones
para distintas entradas

Queremos aplicar la implementación a una función Booleana $f(x_3, x_2, x_1, x_0)$ que es igual a 1 exactamente en las tuplas:

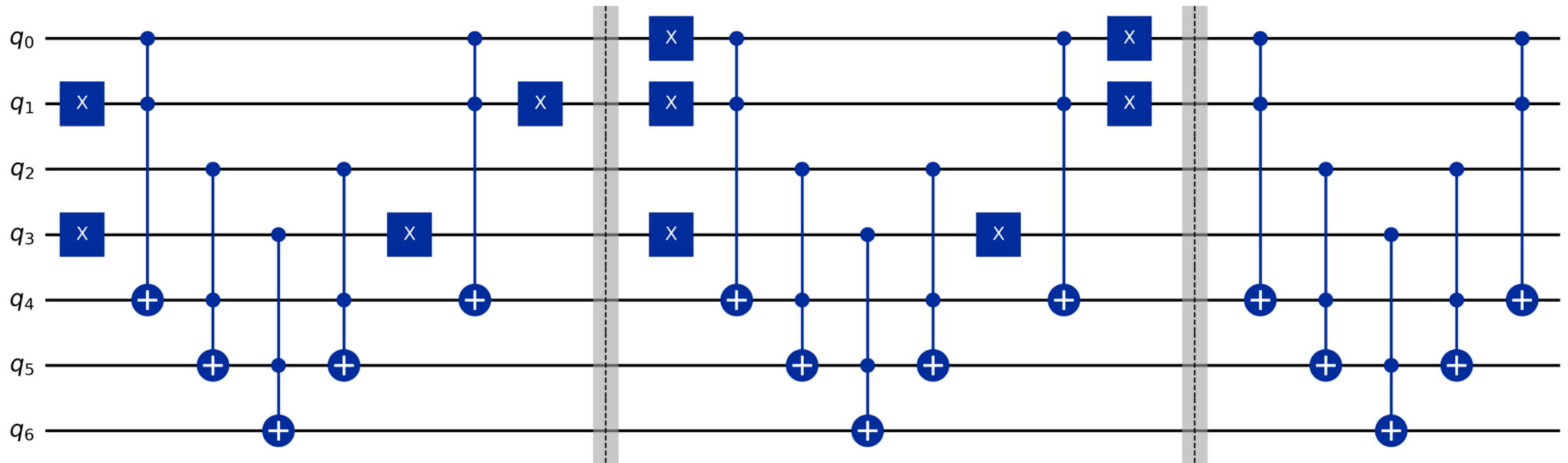
$(0, 1, 0, 1), (0, 1, 0, 0), (1, 1, 1, 1)$

Solución del ejercicio en Qiskit

```
def circuito_tabla(tabla, dim):  
    circuito = QuantumCircuit(2*dim-1)  
    for i in range(len(tabla)):  
        circuito = circuito.compose(circuito_entrada(tabla[i]))  
        if i < len(tabla)-1:  
            circuito.barrier()  
    return circuito
```

```
tabla = [ [0,1,0,1], [0,1,0,0], [1, 1, 1, 1] ]  
dim = 4  
circuito = circuito_tabla(tabla, dim)  
circuito.draw("mpl")
```

Solución del ejercicio en Qiskit



Construcción reversible a partir de un circuito clásico

La construcción de U_f a partir de la tabla de verdad de f toma tiempo exponencial en el peor caso

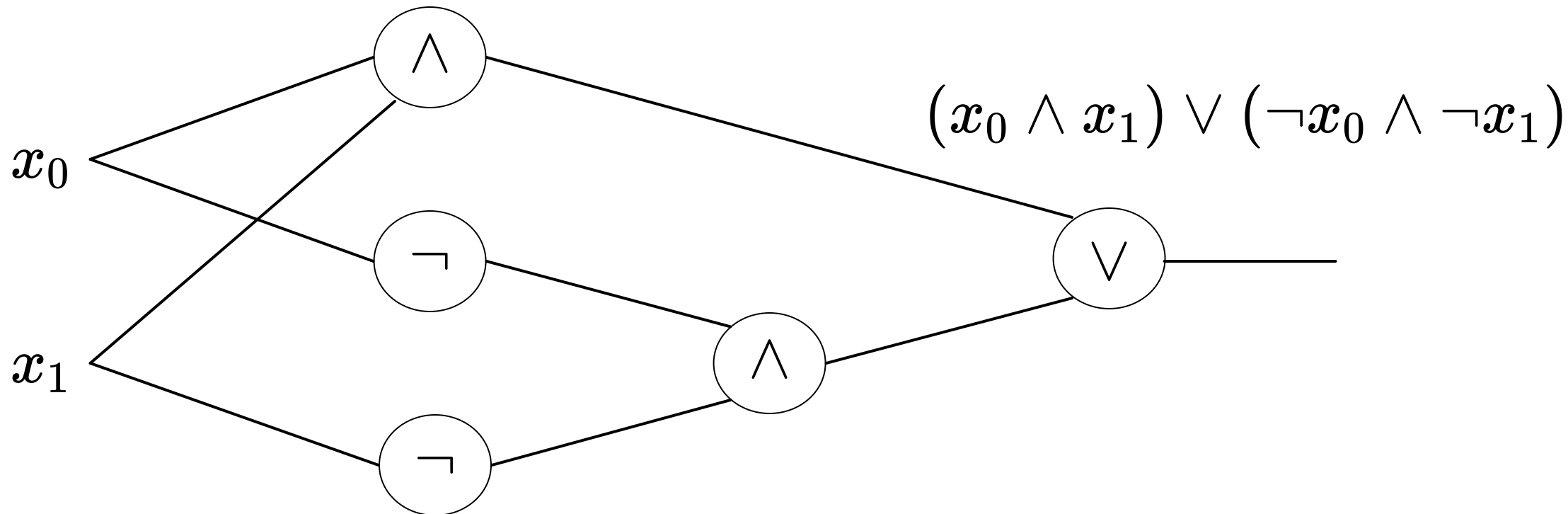
- En general, no podemos usar la construcción anterior en la práctica

Una mejor alternativa es construir U_f a partir de un circuito clásico C_f para la función f

- Suponemos que este circuito usa compuertas $\neg$, $\wedge$ y $\vee$
- En muchos casos tenemos un circuito clásico de tamaño pequeño para especificar una función Booleana

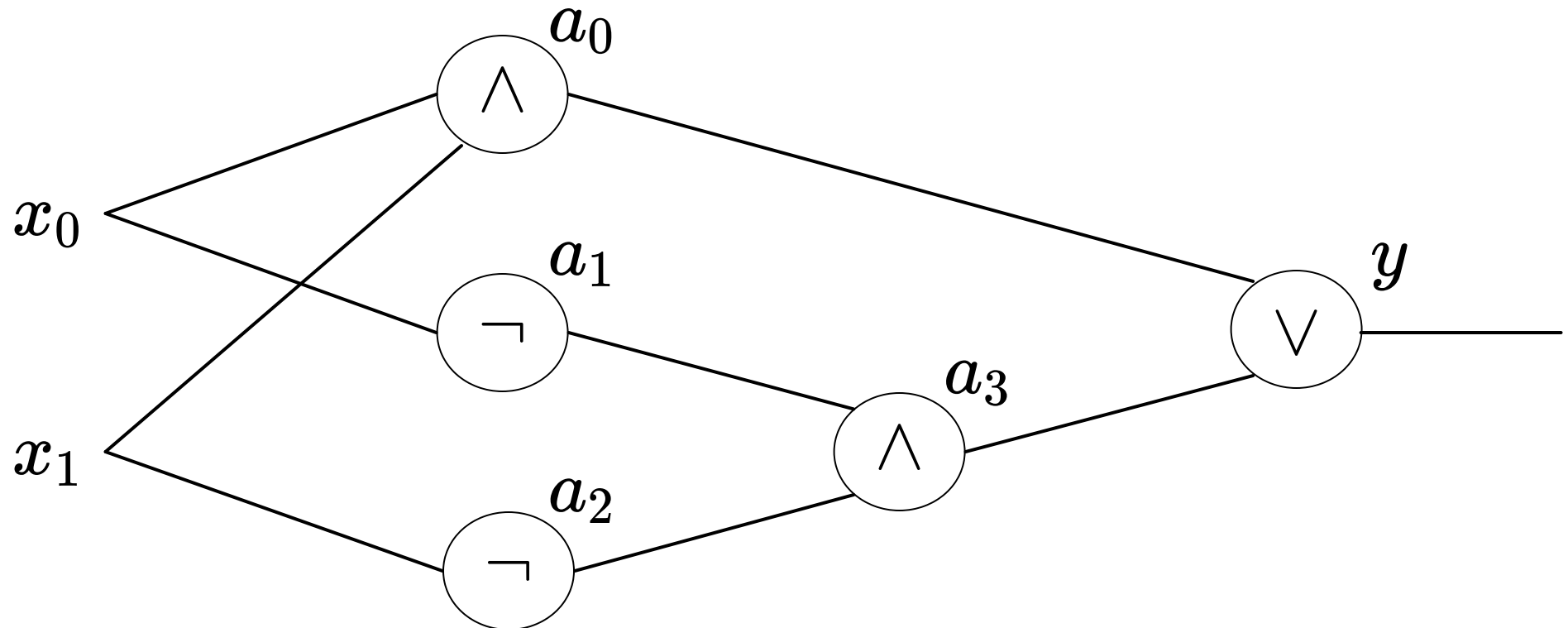
Construcción reversible a partir de un circuito clásico

Considere este ejemplo de circuito clásico que vimos en la introducción:

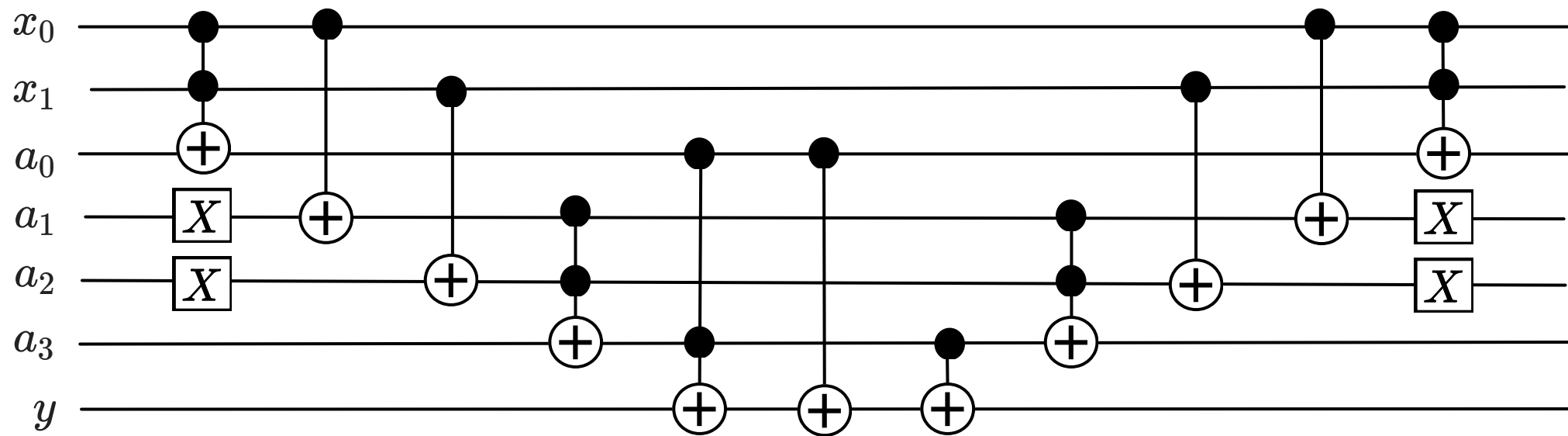


Construcción reversible a partir de un circuito clásico

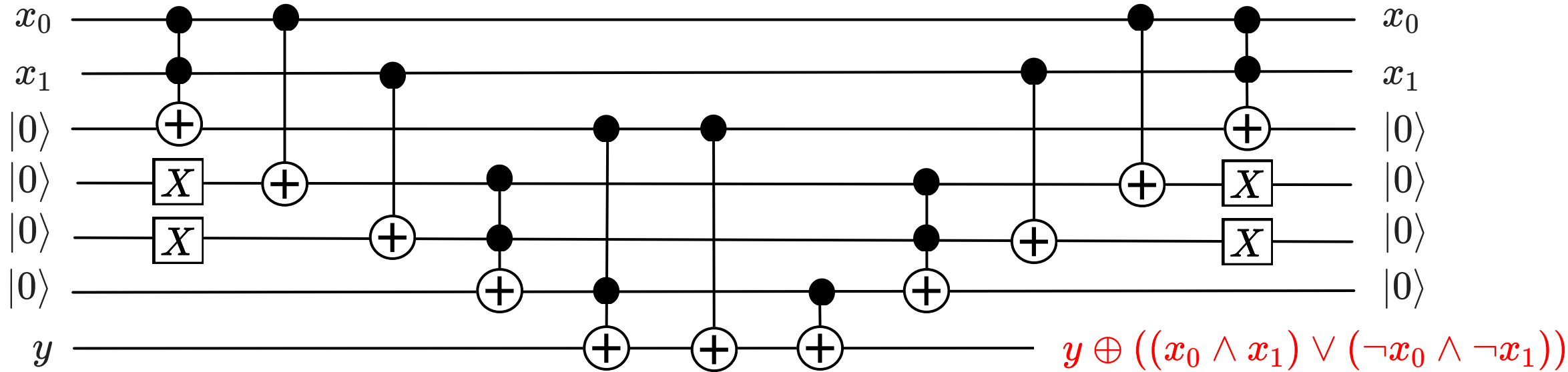
Considere este ejemplo de circuito clásico que vimos en la introducción:



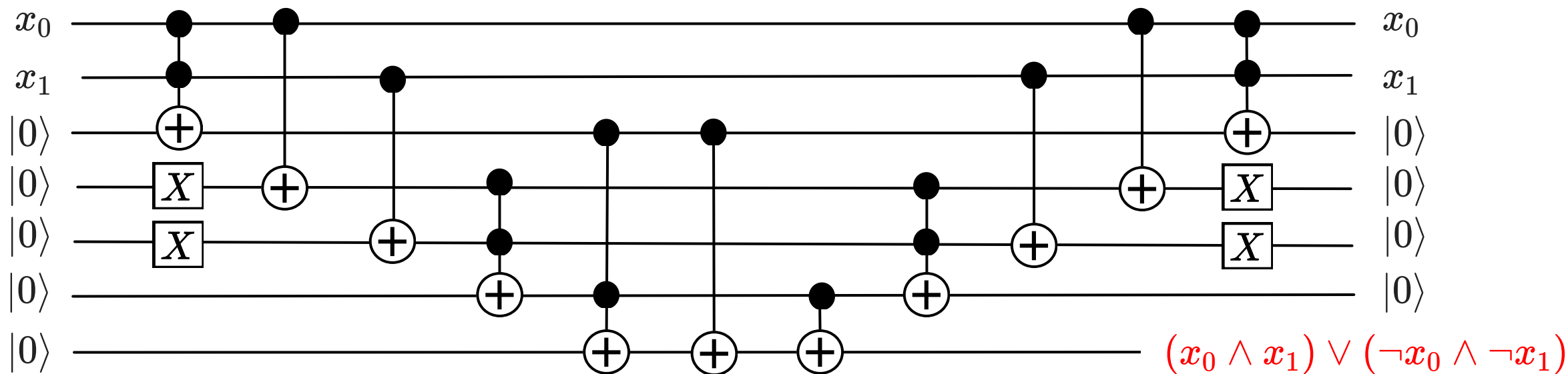
Construcción reversible a partir de un circuito clásico



Construcción reversible a partir de un circuito clásico



Construcción reversible a partir de un circuito clásico



Construcción reversible a partir de un circuito clásico

A partir de un circuito clásico C_f para $f : \{0, 1\}^n \rightarrow \{0, 1\}$, podemos construir un circuito que implementa la transformación unitaria R_f :

$$R_f |y 0^r x\rangle = |(y \oplus f(x)) 0^r x\rangle$$

El costo de construir el circuito que implementa R_f debe considerarse como parte del costo de un algoritmo que utiliza R_f

Construcción reversible a partir de un circuito clásico

Suponga que C_f tiene S compuertas con a lo más dos entradas cada una

- La compuerta $\neg$ tiene una entrada
- Las compuertas $\wedge$ y $\vee$ tienen dos entradas cada una

La construcción reversible utiliza $O(S)$ compuertas y $O(S)$ auxiliares

- Cada ejecución del circuito resultante utiliza $O(S)$ compuertas

Acceso a una lista: índices y datos

Sea L una lista con K elementos, donde cada elemento tiene b bits

- Suponemos que $K = 2^n$
- Los valores n y b pueden ser distintos

Suponemos acceso mediante $U_L(|d\rangle \otimes |i\rangle) = |d \oplus L[i]\rangle \otimes |i\rangle$

- i es un índice de una posición, tiene n bits
- d es un registro de b bits

Acceso a una lista: índices y datos

Sea L una lista con K elementos, donde cada elemento tiene b bits

- Suponemos que $K = 2^n$
- Los valores n y b pueden ser distintos

En particular, para leer el elemento almacenado en la posición i hacemos lo siguiente:

$$U_L(|0^b\rangle \otimes |i\rangle) = |L[i]\rangle \otimes |i\rangle$$

La noción de qRAM

El escenario anterior es implementado mediante la noción de **qRAM**

Una **qRAM** es un sistema de memoria que permite consultar direcciones en superposición conservando su coherencia:

$$U_L \left(|0^b\rangle \otimes \sum_{i=0}^{2^n-1} \alpha_i |i\rangle \right) = \sum_{i=0}^{2^n-1} \alpha_i |L[i]\rangle \otimes |i\rangle$$

Y en general tenemos que:

$$U_L \left(\sum_{j=0}^{2^b-1} \beta_j |j\rangle \otimes \sum_{i=0}^{2^n-1} \alpha_i |i\rangle \right) = \sum_{j=0}^{2^b-1} \sum_{i=0}^{2^n-1} \beta_j \alpha_i (|j \oplus L[i]\rangle \otimes |i\rangle)$$

La noción de qRAM

La QPU prepara los registros cuánticos y utiliza la qRAM para realizar consultas

La qRAM puede implementarse con recursos de la propia QPU o mediante un subsistema dedicado conectado coherentemente a ella

Cada aplicación de U_L cuenta como una consulta que tiene costo 1:

$$U_L \left(\sum_{j=0}^{2^b-1} \beta_j |j\rangle \otimes \sum_{i=0}^{2^n-1} \alpha_i |i\rangle \right)$$

Una función que utiliza la lista

Supongamos que queremos aplicar $g : \{0, 1\}^b \rightarrow \{0, 1\}$ a los elementos de la lista L

- Vale decir, queremos implementar la función $f : \{0, 1\}^n \rightarrow \{0, 1\}$ definida como $f(i) = g(L[i])$

Suponemos además que tenemos un circuito clásico C_g para g

Una función que utiliza la lista

Disponemos entonces de dos recursos para implementar f :

- U_L que permite leer elementos de L
- R_g construido a partir de C_g
 - También se podría tener R_g construido directamente como circuito cuántico

Queremos combinarlos para obtener el oráculo

$$U_f(|y\rangle \otimes |i\rangle) = |y \oplus g(L[i])\rangle \otimes |i\rangle$$

Una función que utiliza la lista

Implementamos la función f de la siguiente forma:

$$\begin{aligned} |y\rangle \otimes |0^r\rangle \otimes |0^b\rangle \otimes |i\rangle &\xrightarrow{U_L} |y\rangle \otimes |0^r\rangle \otimes |L[i]\rangle \otimes |i\rangle \\ &\xrightarrow{R_g} |y \oplus g(L[i])\rangle \otimes |0^r\rangle \otimes |L[i]\rangle \otimes |i\rangle \\ &\xrightarrow{U_L} |y \oplus g(L[i])\rangle \otimes |0^r\rangle \otimes |0^b\rangle \otimes |i\rangle \end{aligned}$$

Una función que utiliza la lista

Vale decir, implementamos una matriz unitaria U_f tal que:

$$U_f(|y\rangle \otimes |0^r\rangle \otimes |0^b\rangle \otimes |i\rangle) = |y \oplus f(i)\rangle \otimes |0^r\rangle \otimes |0^b\rangle \otimes |i\rangle$$

La matriz U_f tiene una dimensión mayor que la que habíamos considerado originalmente

- Esto sucede porque tenemos que usar qubits auxiliares

La matriz U_f se define de la siguiente forma:

$$U_f = \left(I^{\otimes(r+1)} \otimes U_L \right) \left(R_g \otimes I^{\otimes n} \right) \left(I^{\otimes(r+1)} \otimes U_L \right)$$

El costo de un algoritmo cuántico

¿Cuál es el costo de calcular $U_f|\psi\rangle$?

- Dadas U_L y R_g , realizamos dos consultas a U_L y una ejecución de R_g

Cada consulta a U_L tiene costo 1

El costo de un algoritmo cuántico

El costo de ejecutar R_g se paga en cada aplicación del oráculo

- El circuito que implementa R_g puede estar compuesto por varias compuertas

¿Pero no tenemos que considerar también los costos de construir U_L y R_g ?

- U_L está dada, no la construye el algoritmo, por lo tanto **no** se debe incluir el costo de construir U_L
- El costo de construcción de R_g **sí** debe ser considerado

El costo de un algoritmo cuántico

Las consideraciones anteriores son muy importantes al momento de analizar un algoritmo cuántico

Por ejemplo, considere nuevamente una lista L con K elementos, donde cada elemento tiene b bits

Para verificar si $a \in \{0, 1\}^b$ está en L , un algoritmo clásico tiene que realizar K comparaciones en el peor caso

- No suponemos que la lista está ordenada

El costo de un algoritmo cuántico

Sin embargo, vamos a ver que el algoritmo de Grover puede resolver este problema en tiempo $O\left(\sqrt{K}(b + \log K)\right)$ en un computador cuántico, con error acotado

En el análisis de este algoritmo vamos a tener que considerar la construcción del circuito cuántico para $m_a(x)$

- Usamos la operación $m_a(L[i])$ para verificar si el elemento i de la lista es a

El costo de un algoritmo cuántico

El costo de construir un circuito cuántico para $m_a(x)$ es $O(b)$

- En particular, este tiempo no depende de K

Tiene sentido entonces decir que el algoritmo de Grover es más rápido que un algoritmo clásico

- Si el algoritmo de Grover tuviera que construir U_L no lo sería, ya que esta construcción debe considerar los K elementos de L

El oráculo de fase

Para una función Booleana $f : \{0, 1\}^n \rightarrow \{0, 1\}$, el **oráculo de fase** O_f se define de la siguiente forma, suponiendo que $x \in \{0, 1\}^n$:

$$O_f|x\rangle = (-1)^{f(x)}|x\rangle$$

Por lo tanto:

$$O_f \left(\sum_{x \in \{0,1\}^n} \alpha_x |x\rangle \right) = \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \alpha_x |x\rangle$$

Las entradas para las cuales $f(x) = 1$ reciben un signo negativo

¿Existe este oráculo para cualquier función booleana?

Aplicando dos veces la transformación, tenemos que:

$$\begin{aligned} O_f^2 \left(\sum_{x \in \{0,1\}^n} \alpha_x |x\rangle \right) &= O_f \left(\sum_{x \in \{0,1\}^n} \alpha_x (-1)^{f(x)} |x\rangle \right) \\ &= \sum_{x \in \{0,1\}^n} \alpha_x (-1)^{f(x)} (-1)^{f(x)} |x\rangle \\ &= \sum_{x \in \{0,1\}^n} (-1)^{2f(x)} \alpha_x |x\rangle \\ &= \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle \end{aligned}$$

¿Existe este oráculo para cualquier función booleana?

Tenemos que: $O_f^2 = I$

Vamos a demostrar que $O_f^\dagger = O_f$

- De esto concluimos que O_f es una matriz unitaria puesto que $O_f^\dagger O_f = O_f O_f = I$

¿Existe este oráculo para cualquier función booleana?

Considere la matriz: $\sum_{u \in \{0,1\}^n} (-1)^{f(u)} |u\rangle \langle u|$

Tenemos lo siguiente para $x \in \{0,1\}^n$:

$$\begin{aligned} \left(\sum_{u \in \{0,1\}^n} (-1)^{f(u)} |u\rangle \langle u| \right) |x\rangle &= \sum_{u \in \{0,1\}^n} (-1)^{f(u)} |u\rangle \langle u|x\rangle \\ &= \sum_{u \in \{0,1\}^n} (-1)^{f(u)} |u\rangle \langle u|x\rangle = (-1)^{f(x)} |x\rangle = O_f |x\rangle \end{aligned}$$

¿Existe este oráculo para cualquier función booleana?

Concluimos que: $O_f = \sum_{u \in \{0,1\}^n} (-1)^{f(u)} |u\rangle \langle u|$

Por lo tanto:

$$\begin{aligned} O_f^\dagger &= \left(\sum_{u \in \{0,1\}^n} (-1)^{f(u)} |u\rangle \langle u| \right)^\dagger \\ &= \sum_{u \in \{0,1\}^n} ((-1)^{f(u)} |u\rangle \langle u|)^\dagger \\ &= \sum_{u \in \{0,1\}^n} \overline{(-1)^{f(u)}} (|u\rangle \langle u|)^\dagger = \sum_{u \in \{0,1\}^n} (-1)^{f(u)} |u\rangle \langle u| = O_f \end{aligned}$$

¿Existe este oráculo para cualquier función booleana?

Tenemos entonces que la matriz O_f es unitaria

Concluimos que toda función booleana $f : \{0, 1\}^n \rightarrow \{0, 1\}$ admite un oráculo de fase unitario

- f admite un oráculo que puede ser implementado
- ¿Cómo construimos este operador usando las compuertas que conocemos?

De una respuesta a una fase

Vamos a ver cómo obtener O_f a partir de U_f

Suponga que $x \in \{0, 1\}^n$

Tenemos que:

$$\begin{aligned} U_f (|-\rangle \otimes |x\rangle) &= U_f \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \otimes |x\rangle \right) = U_f \left(\frac{|0x\rangle - |1x\rangle}{\sqrt{2}} \right) \\ &= \frac{U_f |0x\rangle - U_f |1x\rangle}{\sqrt{2}} = \frac{|f(x)x\rangle - |(1 \oplus f(x))x\rangle}{\sqrt{2}} = \frac{|f(x)\rangle - |1 \oplus f(x)\rangle}{\sqrt{2}} \otimes |x\rangle \end{aligned}$$

De una respuesta a una fase

Distinguimos dos casos:

$$\frac{|f(x)\rangle - |1 \oplus f(x)\rangle}{\sqrt{2}} = \begin{cases} |-\rangle & f(x) = 0 \\ -|-\rangle & f(x) = 1 \end{cases}$$

Por lo tanto: $U_f(|-\rangle \otimes |x\rangle) = ((-1)^{f(x)}|-\rangle) \otimes |x\rangle = |-\rangle \otimes (-1)^{f(x)}|x\rangle$

De una respuesta a una fase

Por linealidad:

$$U_f \left(|-\rangle \otimes \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle \right) = \sum_{x \in \{0,1\}^n} \alpha_x (-1)^{f(x)} |-\rangle \otimes |x\rangle$$
$$= |-\rangle \otimes \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \alpha_x |x\rangle$$

Así, $U_f (|-\rangle \otimes |\psi\rangle) = |-\rangle \otimes O_f |\psi\rangle$

- El qubit de respuesta $|-\rangle$ queda en el mismo estado, mientras que los valores de la función determinan las fases de $|\psi\rangle$

Construcción reversible a partir de un circuito clásico

Sabemos que desde un circuito clásico C_f para f podemos construir una matriz unitaria R_f tal que $R_f|y0^rx\rangle = |(y \oplus f(x))0^rx\rangle$

Podemos usar R_f para implementar O_f :

$$R_f(|-\rangle \otimes |0^r\rangle \otimes |\psi\rangle) = |-\rangle \otimes |0^r\rangle \otimes O_f|\psi\rangle$$

El problema de Deutsch

Consideremos una función $f : \{0, 1\} \rightarrow \{0, 1\}$

Queremos determinar si la función f es constante o balanceada

- Constante: $f(0) = f(1)$
- Balanceada: $f(0) \neq f(1)$

De manera equivalente, queremos calcular $f(0) \oplus f(1)$

Complejidad clásica del problema de Deutsch

¿Cuántas preguntas sobre f tenemos que hacer en un computador clásico para determinar si es constante o balanceada?

No nos basta sólo con conocer $f(0)$, ni tampoco con conocer $f(1)$

- Necesitamos realizar dos consultas

Complejidad cuántica del problema de Deutsch

¿Cuántos cálculos con U_f tenemos que hacer en un circuito cuántico para determinar si f es constante o balanceada?

El algoritmo de Deutsch muestra que basta con un solo acceso

El oráculo U_f

El oráculo U_f realiza el siguiente cálculo, suponiendo que $x, y \in \{0, 1\}$:

$$U_f|yx\rangle = |(y \oplus f(x))x\rangle$$

Accedemos al oráculo como una caja negra

- De todas formas es útil ver cómo se pueden implementar estos oráculos

El oráculo U_f

Casos constantes:

$$f(0) = f(1) = 0:$$

$$f(0) = f(1) = 1:$$

El oráculo U_f

Casos constantes:

$$f(0) = f(1) = 0:$$

x _____
 y _____

$$f(0) = f(1) = 1:$$

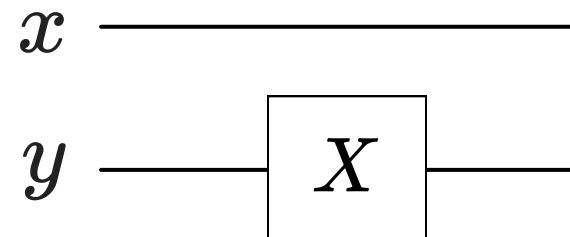
El oráculo U_f

Casos constantes:

$$f(0) = f(1) = 0:$$



$$f(0) = f(1) = 1:$$



El oráculo U_f

Casos balanceados

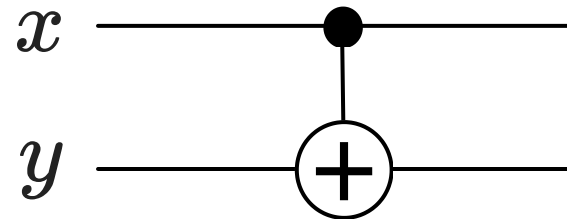
$$f(0) = 0 \text{ y } f(1) = 1:$$

$$f(0) = 1 \text{ y } f(1) = 0:$$

El oráculo U_f

Casos balanceados

$f(0) = 0$ y $f(1) = 1$:

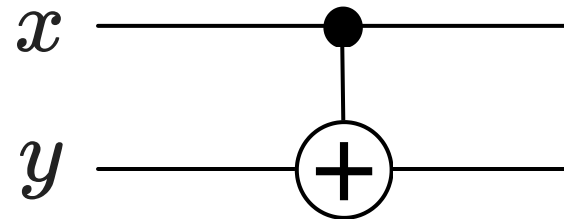


$f(0) = 1$ y $f(1) = 0$:

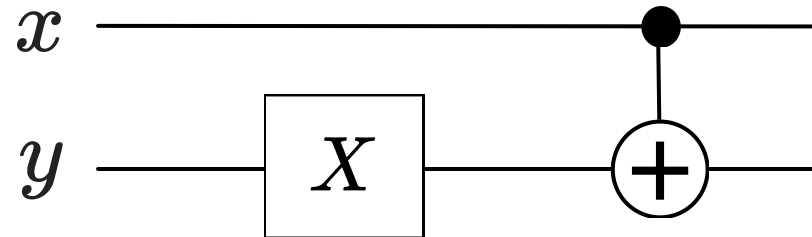
El oráculo U_f

Casos balanceados

$f(0) = 0$ y $f(1) = 1$:

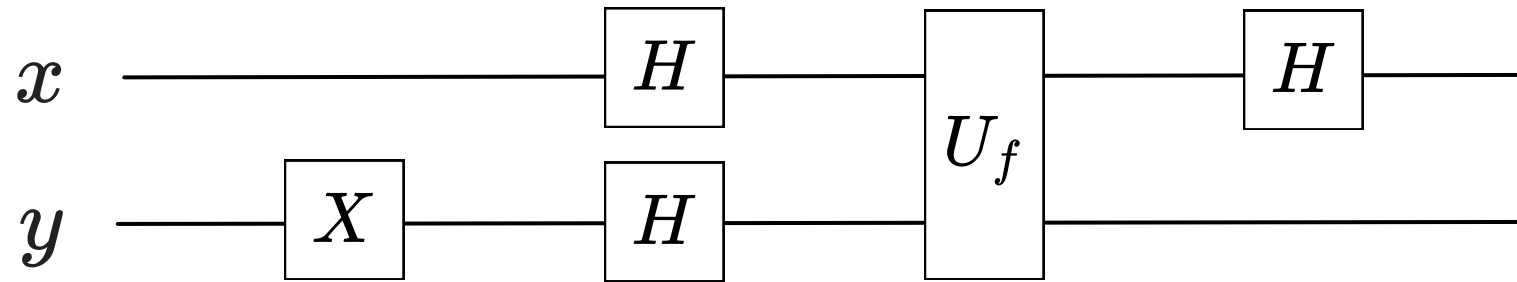


$f(0) = 1$ y $f(1) = 0$:



El circuito de Deutsch

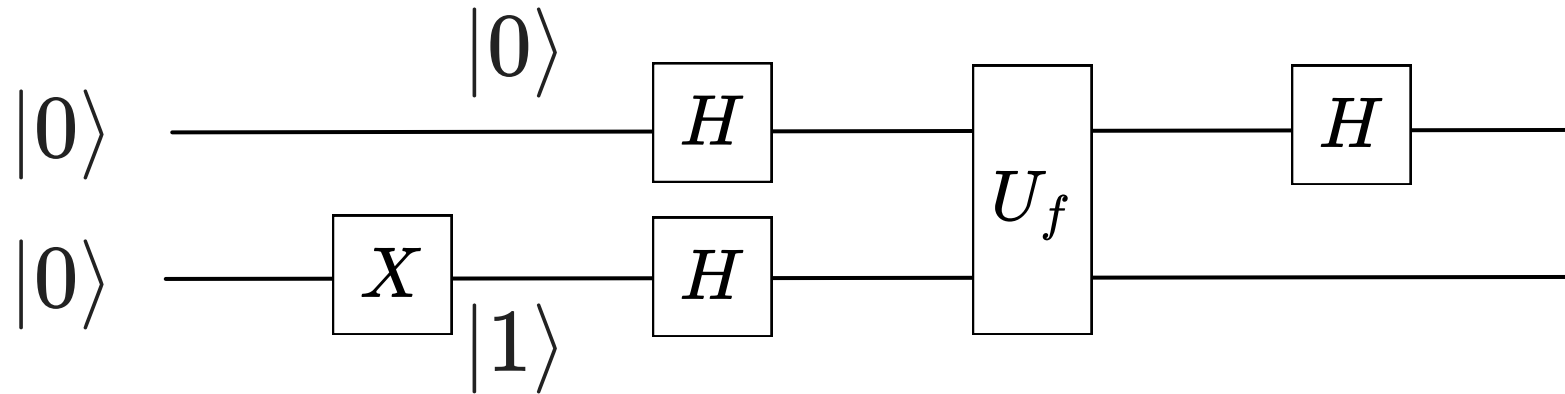
El siguiente circuito resuelve el problema:



La siguiente es la matriz que representa al circuito:

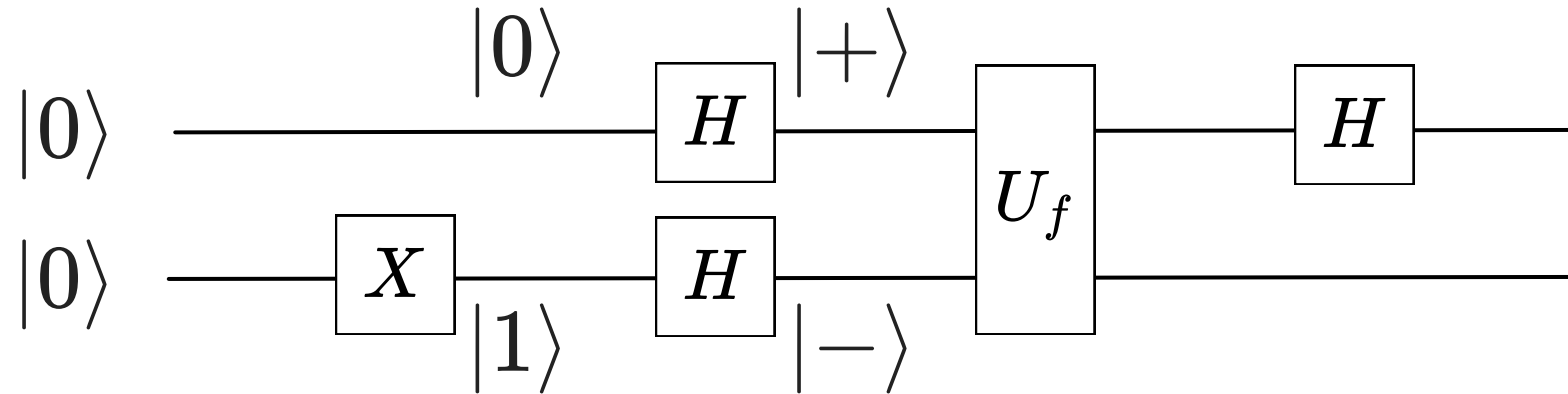
$$D_f = (I \otimes H)U_f(H \otimes H)(X \otimes I)$$

El circuito de Deutsch



$$(X \otimes I)(|0\rangle \otimes |0\rangle) = |1\rangle \otimes |0\rangle$$

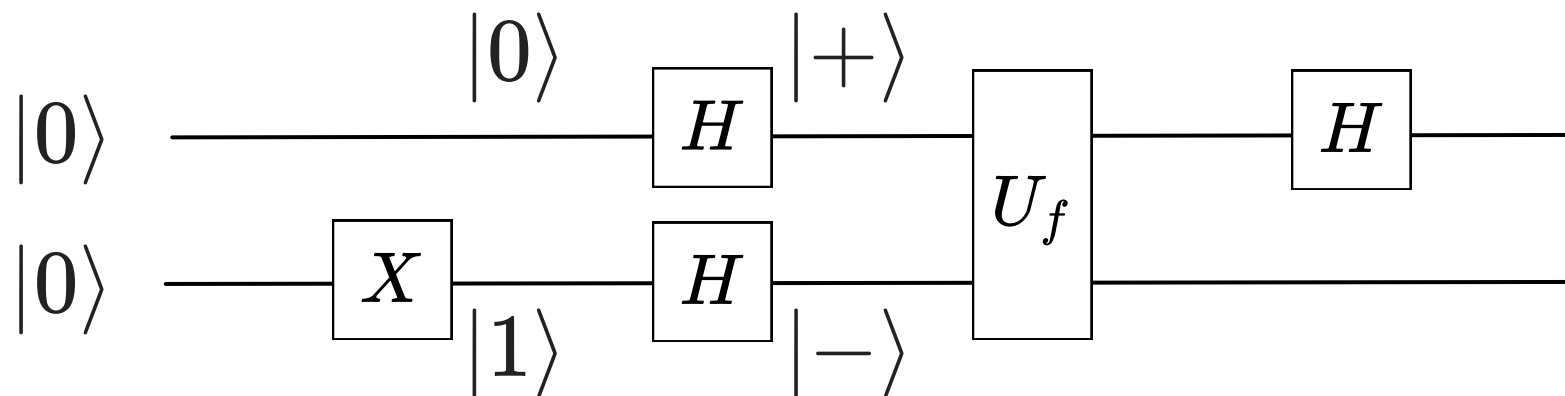
El circuito de Deutsch



$$(H \otimes H)(|1\rangle \otimes |0\rangle) = |-\rangle \otimes |+\rangle$$

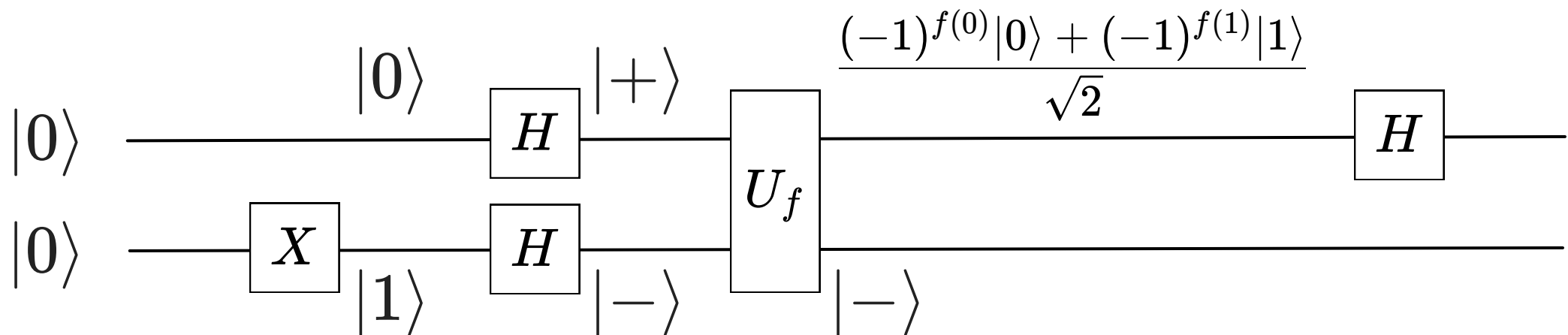
Donde: $|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$ $|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$

El circuito de Deutsch



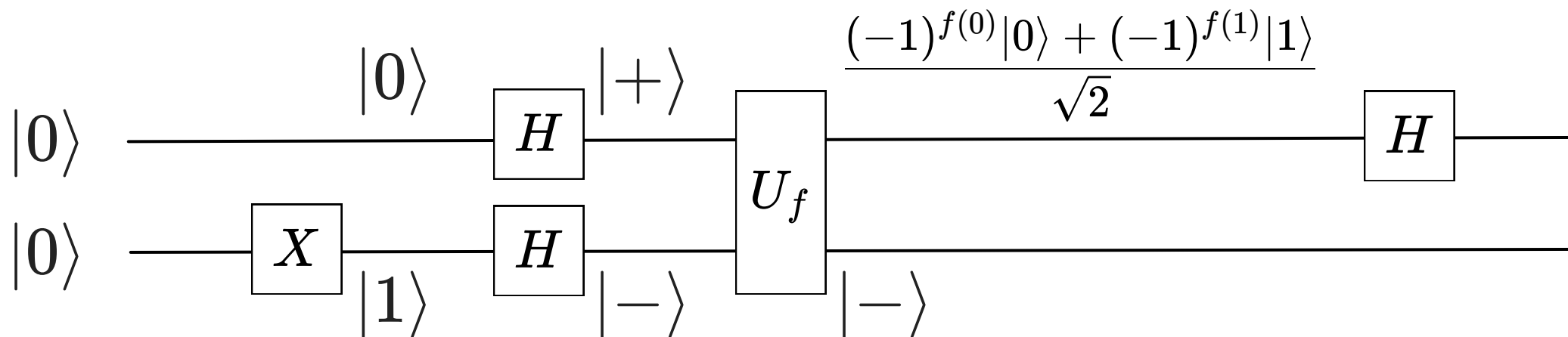
$$U_f(|-\rangle \otimes \frac{|0\rangle + |1\rangle}{\sqrt{2}}) = |-\rangle \otimes \frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}}$$

El circuito de Deutsch



$$U_f(|-\rangle \otimes \frac{|0\rangle + |1\rangle}{\sqrt{2}}) = |-\rangle \otimes \frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}}$$

El circuito de Deutsch



$$(I \otimes H) \left(|-\rangle \otimes \frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}} \right)$$

El circuito de Deutsch

$$\begin{aligned} & (I \otimes H) \left(|-\rangle \otimes \frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}} \right) \\ &= I|-\rangle \otimes H \frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}} \\ &= |-\rangle \otimes \left(\frac{(-1)^{f(0)}}{\sqrt{2}} H|0\rangle + \frac{(-1)^{f(1)}}{\sqrt{2}} H|1\rangle \right) \\ &= |-\rangle \otimes \left(\frac{(-1)^{f(0)}}{\sqrt{2}} \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right) + \frac{(-1)^{f(1)}}{\sqrt{2}} \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right) \right) \\ &= |-\rangle \otimes \left(\frac{(-1)^{f(0)} + (-1)^{f(1)}}{2} |0\rangle + \frac{(-1)^{f(0)} - (-1)^{f(1)}}{2} |1\rangle \right) \end{aligned}$$

El circuito de Deutsch

Por lo tanto, el siguiente es el resultado del circuito:

$$D_f(|0\rangle \otimes |0\rangle) = |-\rangle \otimes \left(\frac{(-1)^{f(0)} + (-1)^{f(1)}}{2} |0\rangle + \frac{(-1)^{f(0)} - (-1)^{f(1)}}{2} |1\rangle \right)$$

El resultado depende de si f es constante o balanceada:

$$f \text{ es constante} \quad \longrightarrow \quad D_f(|0\rangle \otimes |0\rangle) = |-\rangle \otimes (-1)^{f(0)} |0\rangle$$

$$f \text{ es balanceada} \quad \longrightarrow \quad D_f(|0\rangle \otimes |0\rangle) = |-\rangle \otimes (-1)^{f(0)} |1\rangle$$

El circuito de Deutsch

$$f \text{ es constante} \longrightarrow D_f(|0\rangle \otimes |0\rangle) = |-\rangle \otimes (-1)^{f(0)} |0\rangle$$

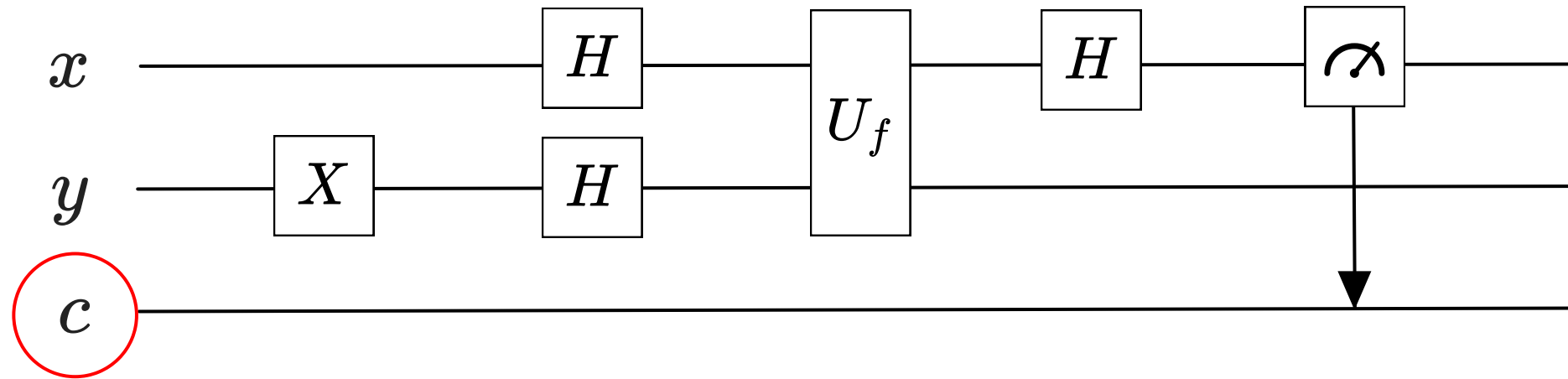
$$f \text{ es balanceada} \longrightarrow D_f(|0\rangle \otimes |0\rangle) = |-\rangle \otimes (-1)^{f(0)} |1\rangle$$

Midiendo el qubit menos significativo podemos saber si f es constante o balanceada

- Solo usamos el oráculo U_f una vez en el cálculo de D_f

El circuito de Deutsch

El siguiente es el circuito completo que resuelve el problema:



bit clásico donde se almacena
el resultado de la medición

El circuito de Deutsch en Qiskit

```
from qiskit import QuantumCircuit
from qiskit.primitives import StatevectorSampler

def oraculo(v0, v1):
    circuito = QuantumCircuit(2)

    if v0 == 0:
        if v1 == 1:
            circuito.cx(0, 1)
    else:
        if v1 == 0:
            circuito.x(1)
            circuito.cx(0, 1)
        else:
            circuito.x(1)

    return circuito
```

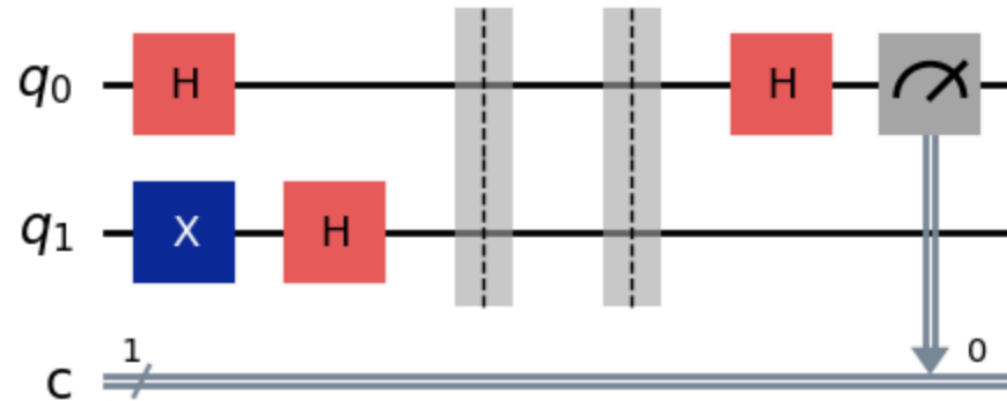
El circuito de Deutsch en Qiskit

```
def circuito_deutsch(oraculo):  
    circuito = QuantumCircuit(2, 1)  
  
    circuito.x(1)  
    circuito.h(0)  
    circuito.h(1)  
    circuito.barrier()  
    circuito.compose(oraculo, inplace=True)  
    circuito.barrier()  
    circuito.h(0)  
    circuito.measure(0, 0)  
  
    return circuito
```

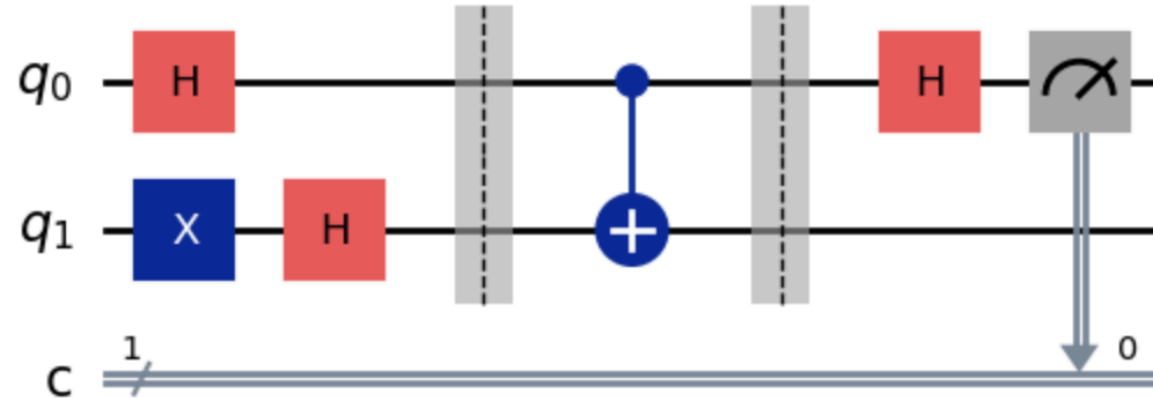
El circuito de Deutsch en Qiskit

```
for v0, v1 in [(0,0), (0,1), (1,0), (1,1)]:  
    U_f = oraculo(v0, v1)  
    D_f = circuito_deutsch(U_f)  
  
    simulador = StatevectorSampler()  
    resultado = simulador.run(  
        [D_f],  
        shots=1000  
    ).result()  
  
    display(D_f.draw("mpl"))  
    conteos = resultado[0].data.c.get_counts()  
    print(conteos)
```

El circuito de Deutsch en Qiskit

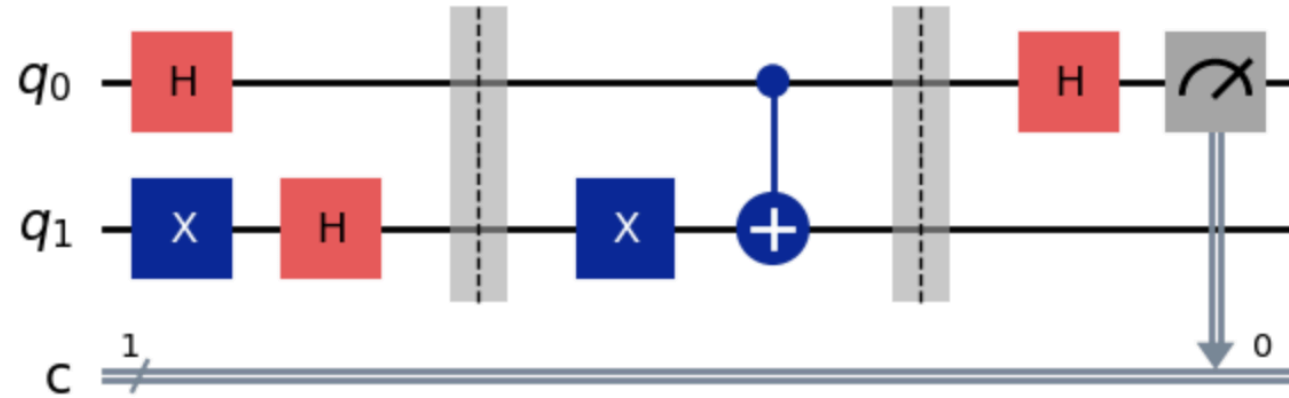


`{'0': 1000}`

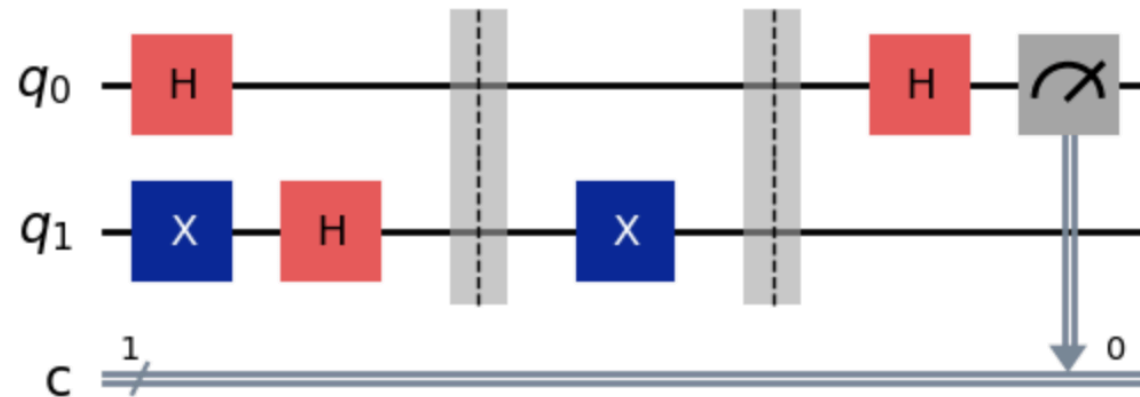


`{'1': 1000}`

El circuito de Deutsch en Qiskit



`{'1': 1000}`



`{'0': 1000}`

El problema de Deutsch–Jozsa

Consideremos ahora una función $f : \{0, 1\}^n \rightarrow \{0, 1\}$

Tenemos la promesa de que f es constante o balanceada

- Constante: $f(x)$ tiene el mismo valor para todo $x \in \{0, 1\}^n$
- Balanceada: $|\{x \in \{0, 1\}^n \mid f(x) = 0\}| = 2^{n-1}$

El objetivo es determinar cuál de los dos casos ocurre

- Para $n = 1$ obtenemos el problema de Deutsch

Complejidad clásica del problema de Deutsch–Jozsa

¿Cuántas preguntas sobre f tenemos que hacer en un computador clásico para determinar si es constante o balanceada?

En el peor caso, necesitamos realizar al menos $2^{n-1} + 1$ consultas si vamos a utilizar un algoritmo determinista

¿Cuántas consultas sobre f se deben hacer si se permite un algoritmo aleatorizado?

- ¿Cuál es el error de este algoritmo?

Complejidad cuántica del problema de Deutsch–Jozsa

¿Cuántos cálculos con U_f tenemos que hacer en un circuito cuántico para determinar si f es constante o balanceada?

El algoritmo de Deutsch–Jozsa muestra que basta con un solo acceso

- Este es un algoritmo determinista

Deutsch–Jozsa tiene una ventaja exponencial en el número de consultas frente a los algoritmos clásicos deterministas

El oráculo U_f

Suponemos que el oráculo U_f realiza el siguiente cálculo, dado $x \in \{0, 1\}^n$ e $y \in \{0, 1\}$:

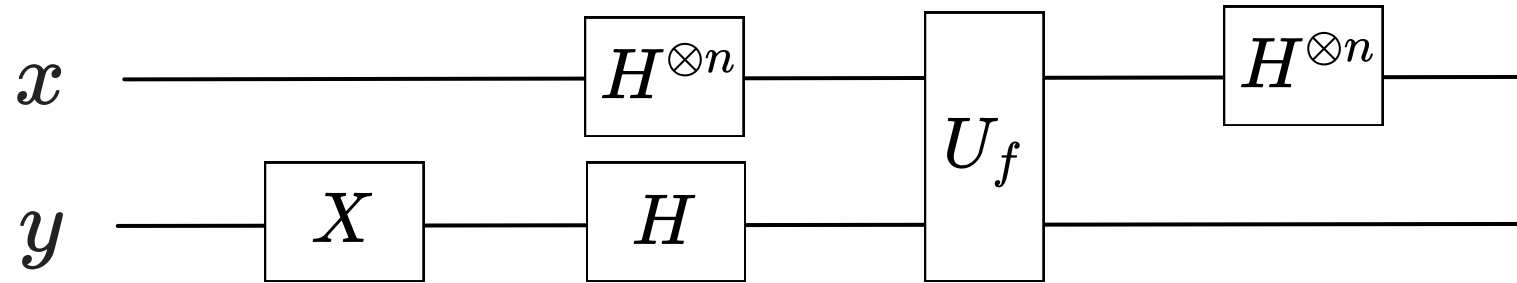
$$U_f|yx\rangle = |(y \oplus f(x))x\rangle$$

Más adelante vamos a discutir cómo se implementa el algoritmo considerando un oráculo U_f con qubits auxiliares:

$$U_f|y0^r x\rangle = |(y \oplus f(x))0^r x\rangle$$

El circuito de Deutsch-Jozsa

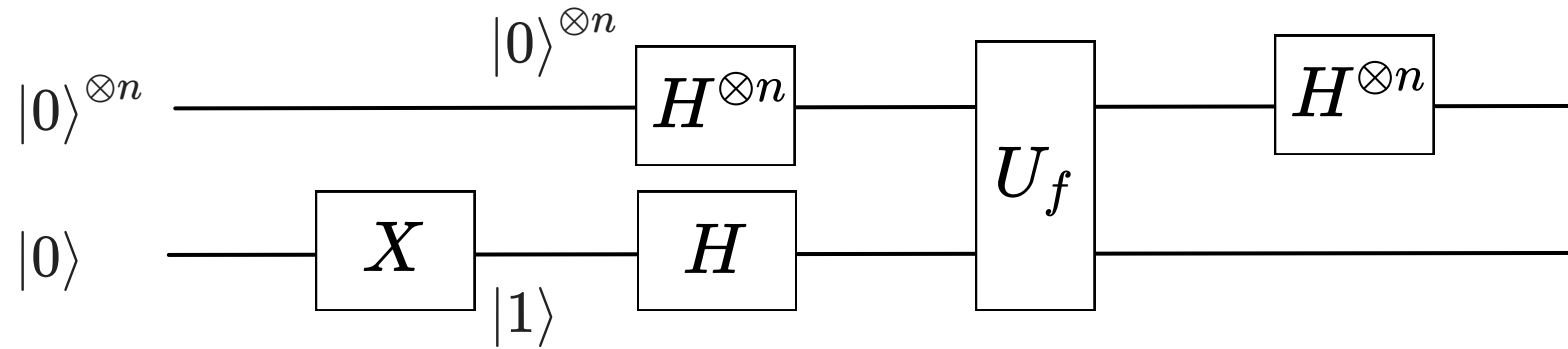
El siguiente circuito resuelve el problema:



La siguiente es la matriz que representa al circuito:

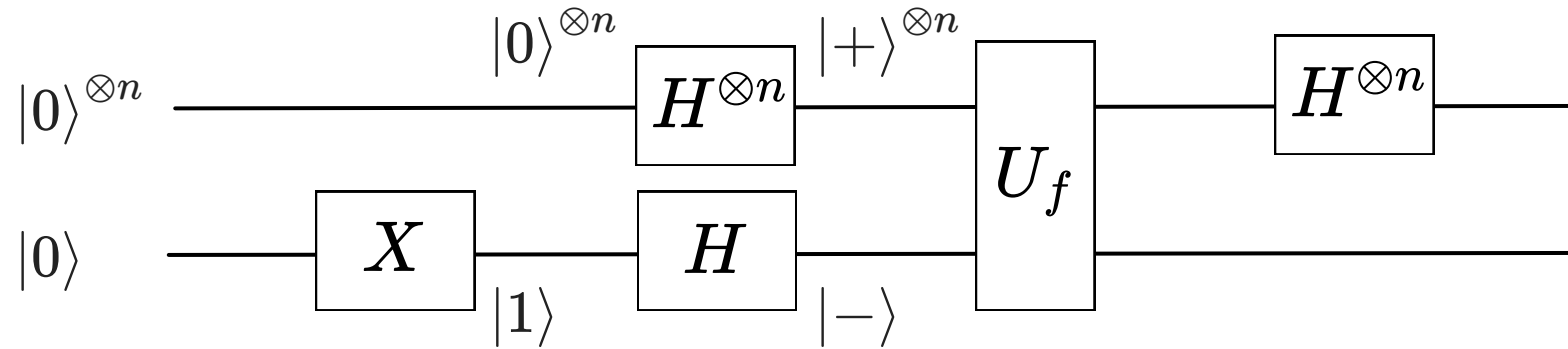
$$D_f = (I \otimes H^{\otimes n})U_f(H \otimes H^{\otimes n})(X \otimes I^{\otimes n})$$

El circuito de Deutsch-Jozsa



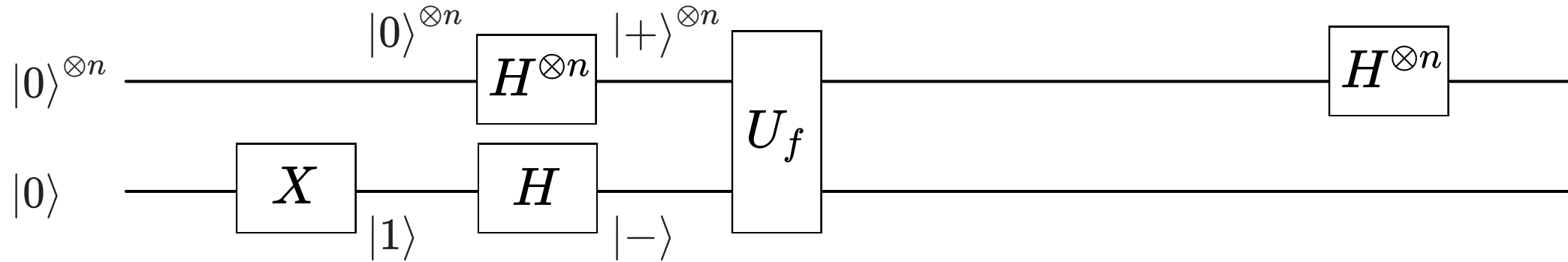
$$(X \otimes I^{\otimes n})(|0\rangle \otimes |0\rangle^{\otimes n}) = |1\rangle \otimes |0\rangle^{\otimes n}$$

El circuito de Deutsch-Jozsa



$$(H \otimes H^{\otimes n})(|1\rangle \otimes |0\rangle^{\otimes n}) = |-\rangle \otimes |+\rangle^{\otimes n}$$

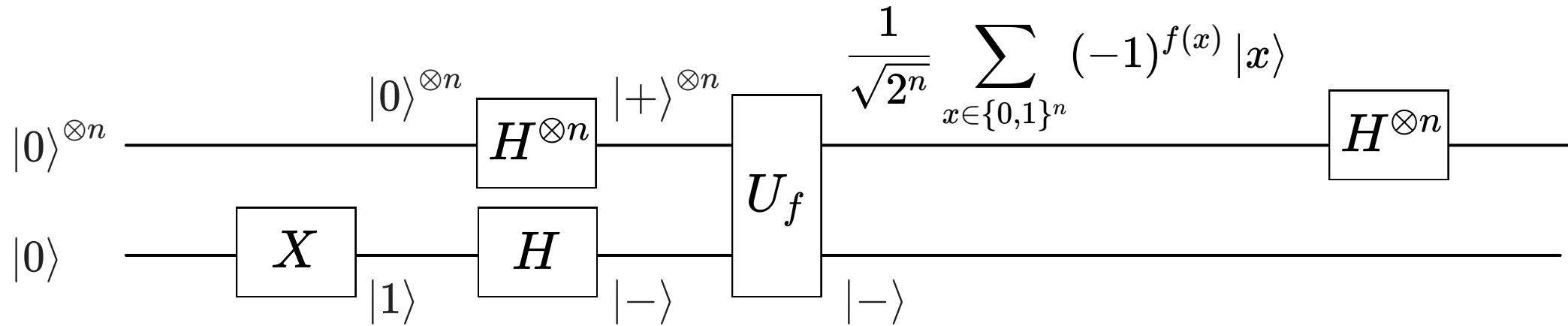
El circuito de Deutsch-Jozsa



$$U_f(|-\rangle \otimes |+\rangle^{\otimes n})$$

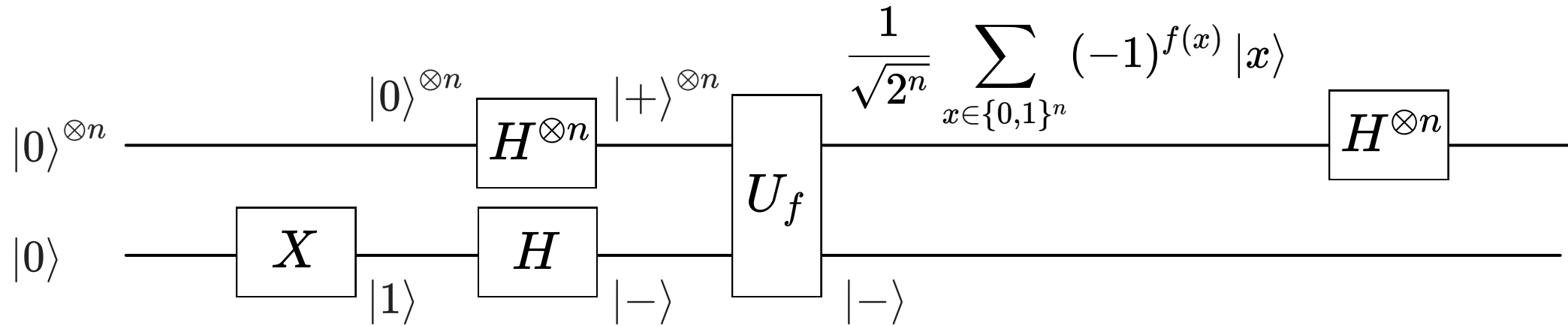
Tenemos que:
$$|+\rangle^{\otimes n} = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right)^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle$$

El circuito de Deutsch-Jozsa



$$\begin{aligned}
 U_f \left(|-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle \right) &= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} U_f(|-\rangle \otimes |x\rangle) \\
 &= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |-\rangle \otimes (-1)^{f(x)} |x\rangle = |-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle
 \end{aligned}$$

El circuito de Deutsch-Jozsa



$$(I \otimes H^{\otimes n}) \left(|-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle \right)$$

El circuito de Deutsch-Jozsa

$$\begin{aligned} (I \otimes H^{\otimes n}) & \left(|-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle \right) \\ &= I |-\rangle \otimes H^{\otimes n} \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle \\ &= |-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} H^{\otimes n} |x\rangle \end{aligned}$$

Tenemos que calcular entonces $H^{\otimes n} |x\rangle$ para $x \in \{0,1\}^n$

El circuito de Deutsch-Jozsa

Suponga que $x = x_{n-1} \cdots x_0$, donde $x_i \in \{0, 1\}$ para cada $i \in \{0, \dots, n-1\}$

Tenemos que:

$$\begin{aligned} H^{\otimes n} |x\rangle &= H^{\otimes n} (|x_{n-1}\rangle \otimes \cdots \otimes |x_0\rangle) \\ &= \bigotimes_{i=1}^n H |x_{n-i}\rangle = \bigotimes_{i=1}^n \frac{|0\rangle + (-1)^{x_{n-i}} |1\rangle}{\sqrt{2}} = \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} (-1)^{z \cdot x} |z\rangle \end{aligned}$$

$$\text{donde } z \cdot x = \bigoplus_{i=0}^{n-1} z_i x_i, \text{ para } z = z_{n-1} \cdots z_0$$

El circuito de Deutsch-Jozsa

Por lo tanto:

$$\begin{aligned} & (I \otimes H^{\otimes n}) \left(|-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle \right) \\ &= |-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} H^{\otimes n} |x\rangle \\ &= |-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \left(\frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} (-1)^{z \cdot x} |z\rangle \right) \\ &= |-\rangle \otimes \frac{1}{2^n} \sum_{x \in \{0,1\}^n} \sum_{z \in \{0,1\}^n} (-1)^{f(x)} (-1)^{z \cdot x} |z\rangle \\ &= |-\rangle \otimes \frac{1}{2^n} \sum_{z \in \{0,1\}^n} \left(\sum_{x \in \{0,1\}^n} (-1)^{f(x) + z \cdot x} \right) |z\rangle \end{aligned}$$

El circuito de Deutsch-Jozsa

Tenemos entonces que si medimos los n qubits menos significativos, la probabilidad de obtener $|0^n\rangle$ es:

$$p = \left(\frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \right)^2$$

Esta probabilidad depende de si f es constante o balanceada:

$$f \text{ es constante} \longrightarrow p = 1$$

$$f \text{ es balanceada} \longrightarrow p = 0$$

El circuito de Deutsch-Jozsa

f es constante $\longrightarrow p = 1$

f es balanceada $\longrightarrow p = 0$

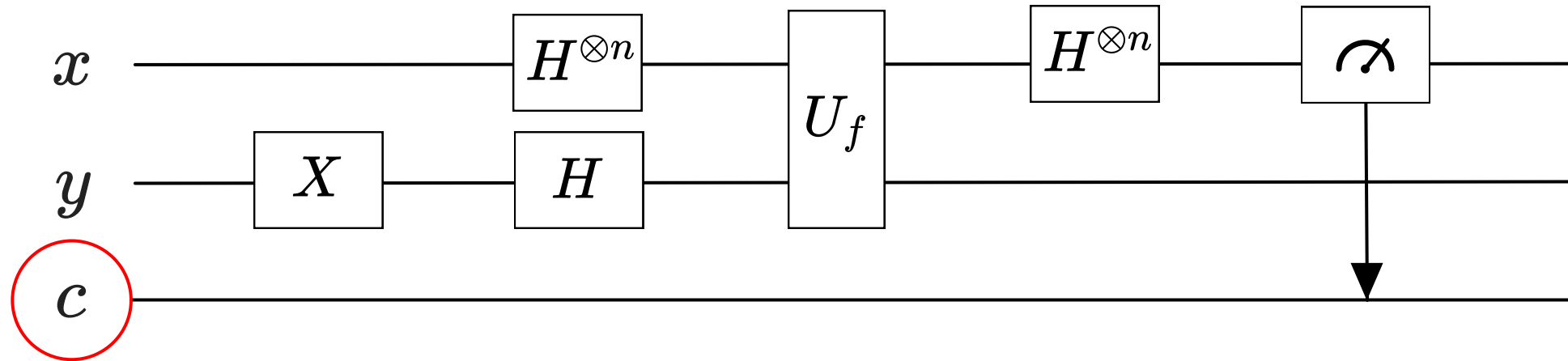
Midiendo los n qubits menos significativos podemos saber si f es constante o balanceada

- Si el resultado es $|0^n\rangle$, entonces f es constante
- Si el resultado es distinto de $|0^n\rangle$, entonces f es balanceada

Solo usamos el oráculo U_f una vez en el cálculo de D_f

El circuito de Deutsch-Jozsa

El siguiente es el circuito completo que resuelve el problema:



n bits clásicos donde se almacena
el resultado de la medición

El circuito de Deutsch-Jozsa

Ejercicio: Resuelva el problema de Deutsch-Jozsa suponiendo que tiene un oráculo U_f para f con qubits auxiliares:

$$U_f|y0^r x\rangle = |(y \oplus f(x))0^r x\rangle$$

- El circuito es esencialmente el mismo que el anterior